



PROGRAMACIÓN ORIENTADA A OBJETOS

TRABAJO PRÁCTICO N°3

1. Escriba un ejemplo del uso de herencia simple, detallando para cada clase cuál es su rol dentro de la herencia y los motivos por los cuales cree que es correcta la implementación. (Trate que el ejemplo sea relacionado a juegos).
2. Explique el concepto de herencia múltiple y cuál es el problema que se presenta al implementarla.
3. Ejemplifique el uso de interfaces y explique si es mejor tener interfaces mas genericas o mas especificas.
4. A partir de las clases Héroe y Enemigo desarrolladas en el TP2, realice un análisis para identificar los atributos y métodos redundantes. Extraiga estos elementos comunes hacia una nueva clase abstracta denominada Personaje, definiendo la visibilidad adecuada para cada atributo junto con sus respectivos métodos getters y setters. Por último, configure las clases Héroe y Enemigo para que hereden de la clase Personaje.
5. En la clase Personaje, desarrolle el método toString() para presentar la información esencial, la cual incluye el nombre y los puntos de vida.
 - a. Lleve a cabo la sobrescritura de este método en la clase Heroe, asegurando que además de los datos de la clase base, se anexen el nivel y los puntos de maná.
6. Defina un método abstracto atacar(Personaje objetivo) en la clase base Personaje. Luego, implemente este método en las clases Enemigo, donde restará su dañoBase a la vida del objetivo, y Héroe, donde deberá restar un daño calculado en función de su nivel.
7. Dentro de la clase Heroe, aplique el concepto de polimorfismo estático a través de la sobrecarga del método atacar, definiendo las siguientes variantes:
 - a. Ataque Básico: public void atacar(Personaje objetivo). Este método ejecuta una ofensiva física estándar que resta de la salud del oponente un daño base calculado como: nivel * 5.
 - b. Ataque Especial con Modificador: public void atacar(Personaje objetivo, double multiplicadorFuerza). Simula un impacto crítico o el uso de una habilidad física avanzada. El daño infligido que se debe restar al objetivo se calcula como: (nivel * 5) * multiplicadorFuerza.
8. Con el fin de experimentar el polimorfismo dinámico, defina en la clase abstracta Personaje la firma del método: public void presentarse()
 - a. Implemente este método en dicha clase base para que muestre en pantalla: "Soy el personaje [nombre] y estoy listo para la acción."
 - b. Realice la sobrescritura (@Override) de este método en la clase Heroe para que imprima: "¡Soy el Héroe [nombre] y lucharé por la justicia! (Vida: [puntosVida])".



PROGRAMACIÓN ORIENTADA A OBJETOS

TRABAJO PRÁCTICO N°3

- c. Lleve a cabo también la sobrescritura (@Override) en la clase Enemigo para que devuelva: "¡Soy el temible [nombre] y voy a destruirte! (Vida: [puntosVida])"
9. Implemente dos nuevas clases derivadas de Héroe:
 - a. Mago: agregue la variable poderMagico y redefina el método atacar(Personaje objetivo) restando maná para ejecutar un hechizo.
 - b. Guerrero: incorpore la propiedad escudo y sobrescriba recibirDaño(Integer cantidad) para que el daño recibido sea mitigado parcialmente por el escudo antes de afectar su salud.
10. Dentro del método main, programe una simulación de combate masivo que emplee un solo arreglo de tipo Personaje[] (o bien un ArrayList<Personaje>) en el cual coexistan instancias de Héroe, Magos, Guerreros y Enemigos.
 - a. Realice la carga inicial de la estructura combinando las diferentes clases de personajes disponibles.
 - b. Itere sobre dicha colección mediante un bucle para que cada integrante ejecute su método atacar() sobre un oponente seleccionado de forma aleatoria dentro de la misma lista.